Milestone Four Narrative

Enhancement Three: Databases

Garrett Sikes
9/28/2026
SNHU
CS 499 Computer Science Capstone

Artifact: Travlr Getaways, database layer (Mongoose schema, trips controller, and JWT middleware)

This artifact is a continuation of the Travlr Getaways full-stack travel booking site I built in Full Stack Development I (CS 465). This piece focuses specifically on the database layer: the MongoDB schema defined through Mongoose, the trips controller, and the JWT authentication middleware that protects it.

When writing code for an assignment, there is always a deadline, and that sometimes means the finished product is just good enough to submit rather than something I would consider solid. I chose to revisit this project for two reasons: it was still fresh in my mind, since I had just come off the previous semester with it as my final project, and I knew it was exactly that kind of good enough work, so being given the chance to go back and enhance something I had already built felt like the right opportunity to take. The clearest weakness I found was in the authentication middleware protecting the trip endpoints. The route for deleting a trip had no login check applied to it at all, so anyone could delete a trip without ever authenticating. On the routes that were supposed to check for a valid token, the authentication function had a bug that let any request through to the next step no matter what the token check actually found; it appeared to work correctly, but only because of an unrelated coding mistake elsewhere in the same function that happened to crash bad requests before they went through, not because the logic itself was sound. I rewrote the function so it only allows a request to continue once the token is actually confirmed valid, and applied that same check to the delete route, so deleting a trip now requires that same valid-token check already used for adding and editing one.

Alongside that fix, I also cleaned up the schema itself: I added trim validation to strip stray whitespace from text fields, and changed the perPerson field from a plain string to a number, so the database now enforces that a trip's price is an actual numeric value instead of accepting whatever gets typed into it.

In my Module One plan, all three of my chosen enhancements were built around one theme: that security is not a checkbox, it is part of how I build things from the start. For the Travlr database specifically, that meant real schema validation and making sure nothing let bad data reach the database, which lines up directly with the program outcome of developing a security mindset that anticipates adversarial exploits and mitigates design flaws. I went into this enhancement with that mindset, found real flaws in my own code, and closed them. In a way I did not expect when I started, this enhancement also ended up satisfying a second outcome: using well-founded and innovative techniques and tools in computing practices to implement solutions. What began as a

security-first fix became just as much about making the database layer function correctly, so the finished result is both secure and functional.

It is hard sometimes to look at your own code critically and ask what could still be improved, especially after a few passes where you feel like you already got it right. That was exactly how I felt going back into this project, which made finding the authentication bug an eye-opening moment. I did not run into any major roadblocks working through this enhancement; I was mainly just surprised that a flaw that significant had been sitting there overlooked, since the mistake causing it was so minor while the security flaw it created was so monumental.